```
  x => x.DisableDataAnnotationsValidation = true)                    ❷
    .AddFluentValidationClientsideAdapters();                        ❸

ValidatorOptions.Global.LanguageManager.Enabled = false;             ❹
```

❶ Instead of manually registering validators, FluentValidation can autoregister them for you.

❷ Setting to true disables DataAnnotations validation completely for model binding.

❸ Enables integration with client-side validation via data-* attributes

❹ FluentValidation has full localization support, but you can disable it if you don't need it.

It's important to understand that if you don't set `DisableDataAnnotationsValidation` to `true`, ASP.NET Core will run validation with both `DataAnnotations` and FluentValidation. That may be useful if you're in the process of migrating from one system to the other, but otherwise, I recommend disabling it. Having your validation split between both places seems like the worst of both worlds!

One final thing to consider is where to put your validators in your solution. There are no technical requirements for this; if you've registered your validator with the DI container, it will be used correctly, so the choice is up to you. I prefer to place validators close to the models they're validating.

For Razor Pages binding-model validators, I create the validator as a nested class of the `PageModel`, in the same place as I create the `InputModel`, as described in chapter 16. That gives a class hierarchy in the Razor Page similar to the following:

```
public class IndexPage : PageModel
{
    public class InputModel { }
    public class InputModelValidator: AbstractValidator<InputModel> { }
}
```